

Introduction

C is a programming language developed at AT & T's Bell Laboratories of USA in 1972. It was designed and written by a man named Dennis Ritchie. In the late seventies C began to replace the more familiar languages of that time like PL/I, ALGOL, etc.

ANSI C standard emerged in the early 1980s, this book was split into two titles: The original was still called Programming in C, and the title that covered ANSI C was called Programming in ANSI C. This was done because it took several years for the compiler vendors to release their ANSI C compilers and for them to become ubiquitous. It was initially designed for programming UNIX operating system. Now the software tool as well as the C compiler is written in C. Major parts of popular operating systems like Windows, UNIX, Linux is still written in C. This is because even today when it comes to performance (speed of execution) nothing beats C. Moreover, if one is to extend the operating system to work with new devices one needs to write device driver programs. These programs are exclusively written in C. C seems so popular is because it is reliable, simple and easy to use. often heard today is – “C has been already superseded by languages like C++, C# and Java.

Program:

There is a close analogy between learning English language and learning C language. The classical method of learning English is to first learn the alphabets used in the language, then learn to combine these alphabets to form words, which in turn are combined to form sentences and sentences are combined to form paragraphs. Learning C is similar and easier. Instead of straight-away learning how to write programs, we must first know what alphabets, numbers and special symbols are used in C, then how using them constants, variables and keywords are constructed, and finally how are these combined to form an instruction. A group of instructions would be combined later on to form a program. So, a computer program is just a collection of the instructions necessary to solve a specific problem. The basic operations of a computer system form what is known as the computer's instruction set. And the approach or method that is used to solve the problem is known as an algorithm. So, for as programming language concern these are of two types.

- 1) Low level language
- 2) High level language

Low level language:

Low level languages are machine level and assembly level language. In machine level language computer only understand digital numbers i.e. in the form of 0 and 1. So, instruction given to the computer is in the form binary digit, which is difficult to implement instruction in binary code. This type of program is not portable, difficult to maintain and also error prone. The assembly language is on other hand modified version of machine level language. Where instructions are given in English like word as ADD, SUM, MOV etc. It is easy to write and understand but not understand by the machine. So the translator used here is assembler to translate into machine level. Although language is bit easier, programmer has to know low level details related to low level language. In the assembly level language the data are stored in the computer register, which varies for different computer. Hence it is not portable.

High level language:

These languages are machine independent, means it is portable. The language in this category is Pascal, Cobol, Fortran etc. High level languages are understood by the machine. So, it needs to translate by the translator into machine level. A translator is software which is used to translate high level language as well as low level language in to machine level language.

Three types of translators are there:

Compiler

Interpreter

Assembler

Compiler and interpreter are used to convert the high-level language into machine level language. The program written in high level language is known as source program and the corresponding machine level language program is called as object program. Both compiler and interpreter perform the same task but there working is different. Compiler read the program at-a-time and searches the error and lists them. If the program is error free then it is converted into object program. When program size is large then compiler is preferred. Whereas interpreter read only one line of the source code and convert it to object code. If it checks error, statement by statement and hence of take more time.

Integrated Development Environments (IDE):

The process of editing, compiling, running, and debugging programs is often managed by a single integrated application known as an Integrated Development Environment, or IDE for short. An IDE is a windows-based program that allows us to easily manage large software programs, edit files in windows, and compile, link, run, and debug programs.

On Mac OS X, CodeWarrior and Xcode are two IDEs that are used by many programmers. Under Windows, Microsoft Visual Studio is a good example of a popular IDE. Kylix is a popular IDE for developing applications under Linux. Most IDEs also support program development in several different programming languages in addition to C, such as C# and C++.

Structure of C Language program:

1) Comment line

2) Preprocessor directive

3) Global variable declaration

4) main function()

{

Local variables;

Statements;

}

User defined function

}

}

Comment line:

It indicates the purpose of the program. It is represented as:

```
/* ..... */
```

Comment line is used for increasing the readability of the program. It is useful in explaining the program and generally used for documentation. It is enclosed within the decimeters. Comment line can be single or multiple line but should not be nested. It can be anywhere in the program except inside string constant & character constant.

Preprocessor Directive:

#include<stdio.h> tells the compiler to include information about the standard input/output library. It is also used in symbolic constant such as #define PI 3.14(value). The stdio.h (standard input output header file) contains definition & declaration of system defined function such as printf(), scanf(), pow() etc. Generally printf() function used to display and scanf() function used to read value

Global Declaration:

This is the section where variables are declared globally so that it can be access by all the functions used in the program. And it is generally declared outside the function.

main()

It is the user defined function and every function has one main() function from where actually program is started and it is enclosed within the pair of curly braces. The main() function can be anywhere in the program but in general practice it is placed in the first position.

Syntax:

```
main()  
{  
.....  
.....  
.....  
}
```

The main() function return value when it declared by data type as

```
int main( )  
{  
return 0  
}
```

The main function does not return any value when void (means null/empty) as void main(void) or void main()

```
{
```

```
printf ("C language");  
}
```

Output: C language

The program execution starts with opening braces and end with closing brace. And in between the two braces declaration part as well as executable part is mentioned. And at the end of each line, the semi-colon is given which indicates statement termination.

```
/*First c program with return statement*/  
  
#include <stdio.h>  
  
int main (void)  
{  
  
printf ("welcome to c Programming language.\n");  
  
return 0;  
}
```

Output: welcome to c programming language.

Steps for Compiling and executing the Programs:

There are four steps in this process.

1. Writing and editing the program using Text editor (source code).
2. Compile the program using any C compiler.(.bak file)
3. Linking the program with the required library modules(object file)
4. Executing the program. (.Exe file)

Identifiers:

Identifiers refer to the names of variables, constants, functions and arrays. These are user-defined names is called Identifiers. These identifier are defined against a set of rules.

Rules for an Identifier

1. An Identifier can only have alphanumeric characters(a -z , A-Z , 0-9) and underscore(_).
- 2.The first character of an identifier can only contain alphabet(a-z , A-Z) or underscore (_).
- 3.Identifiers are also case sensitive in C. For example name and Name are two different identifier in C.
- 4.Keywords are not allowed to be used as Identifiers.
- 5.No special characters, such as semicolon, period, whitespaces, slash or comma are permitted to be used in or as Identifier.
- 6.C" compiler recognizes only the first 31 characters of an identifiers

Keywords:

A keyword is a reserved word. All keywords have fixed meaning that means we cannot change. Keywords serve as basic building blocks for program statements. All keywords must be written in lowercase.

Data Types

Each variable in C has an associated data type. The type of a variable determines how much space it occupies in storage and how the bit pattern stored is interpreted. Each data type requires different amounts of memory and has some specific operations which can be performed over it.

Type	Storage Size	Value
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
Int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
Unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
Short	2 bytes	-32,768 to 32,767
Unsigned short	2 bytes	0 to 65,535
Long	8 bytes	-9223372036854775808 to 9223372036854775807
Unsigned long	8 bytes	0 to 18446744073709551615
float	4 bytes	1.2E-38 to 3.4E+38
double	8 bytes	2.3E-308 to 1.7E+308

Variables:

In programming, a variable is a container (storage area) to hold data. To indicate the storage area, each variable should be given a unique name (identifier). Variable names are just the symbolic representation of a memory location. When a variable is defined, it is not initialized. We must initialize any variable requiring prescribed data when the function starts.

Variable Declaration

All variables must be declared before use, although certain declarations can be made implicitly by context. A declaration specifies a type, and contains a list of one or more variables of that type, as in

```
int lower, upper, step;
```

Constants

Constants refer to fixed values that do not change during the execution of a program

Two ways to define constant in C

There are two ways to define constant in C programming.

1.const keyword

2.#define preprocessor

1) C const keyword

The const keyword is used to define constant in C programming.

Eg. const float PI=3.14;

2) C #define preprocessor

The #define preprocessor directive is used to define constant or micro substitution. It can use any basic data type.

Syntax: #define token value

Eg. #define PI 3.14

OPERATORS

An expression is any valid set of literals, variables, operators, operands and expressions that evaluates to a single value. This value can be a number, a string or a logical value.

For instance, $a = b + c$; denotes an expression in which there are 3 operands a , b , c and two operators $+$ and $=$. A statement, the smallest independent computational unit, specifies an action to be performed.

Arithmetic Operators

The binary arithmetic operators are $+$, $-$, $*$, $/$, and the modulus operator $\%$. Integer division truncates any fractional part.

Relational and Logical Operators

The relational operators are

$>$ $>=$ $<$ $<=$

Relational operators have lower precedence than arithmetic operators, so an expression like $i < j-1$ is taken as $i < (j-1)$, as would be expected.

Similarly, the precedence of $\&\&$ is higher than that of $||$, and both are lower than relational and assignment operators.

Increment and Decrement Operators

C provides two unusual operators for incrementing and decrementing variables. The increment operator $++$ adds 1 to its operand, while the decrement operator $--$ subtracts 1.

Bitwise Operators

C provides six operators for bit manipulation; these may only be applied to integral operands, that is, `char`, `short`, `int`, and `long`, whether signed or unsigned.

	bitwise AND
	bitwise inclusive OR
	bitwise exclusive OR
$<<$	left shift
$>>$	right shift
	one's complement (unary)

Assignment Operators and Expressions

Expressions such as

$i = i + 2$;

in which the variable on the left-hand side is repeated immediately on the right, can be written in the compressed form

$i += 2$;

The operator $+=$ is called an *assignment operator*.

Type Conversions

When an operator has operands of different types, they are converted to a common type according to a small number of rules. In general, the only automatic conversions are those that convert a "narrower" operand into a "wider" one without losing information, such as converting an integer to floating point in an expression like $f + i$. Expressions that don't

make sense, like using a **float** as a subscript, are disallowed. Expressions that might lose information, like assigning a longer integer type to a shorter, or a floating-point type to an integer, may draw a warning, but they are not illegal.

Input and Output

In “C” language, two types of Input/Output functions are available, and all input and output operations are carried out through function calls. Several functions are available for input / output operations in “C”. These functions are collectively known as the standard i/o library.

Input: In any programming language input means to feed some data into program. This can be given in the form of file or from command line.

Output: In any programming language output means to display some data on screen, printer or in any file.

The model of input and output supported by the standard library is very simple. Text input or output, regardless of where it originates or where it goes to, is dealt with as streams of characters. A *text stream* is a sequence of characters divided into lines; each line consists of zero or more characters followed by a newline character. It is the responsibility of the library to make each input or output stream conform to this model; the C programmer using the library need not worry about how lines are represented outside the program.

The standard library provides several functions for reading or writing one character at a time, of which **getchar** and **putchar** are the simplest. Each time it is called, **getchar** reads the *next input character* from a text stream and returns that as its value. That is, after

```
c = getchar()
```

the variable **c** contains the next character of input. The characters normally come from the keyboard.

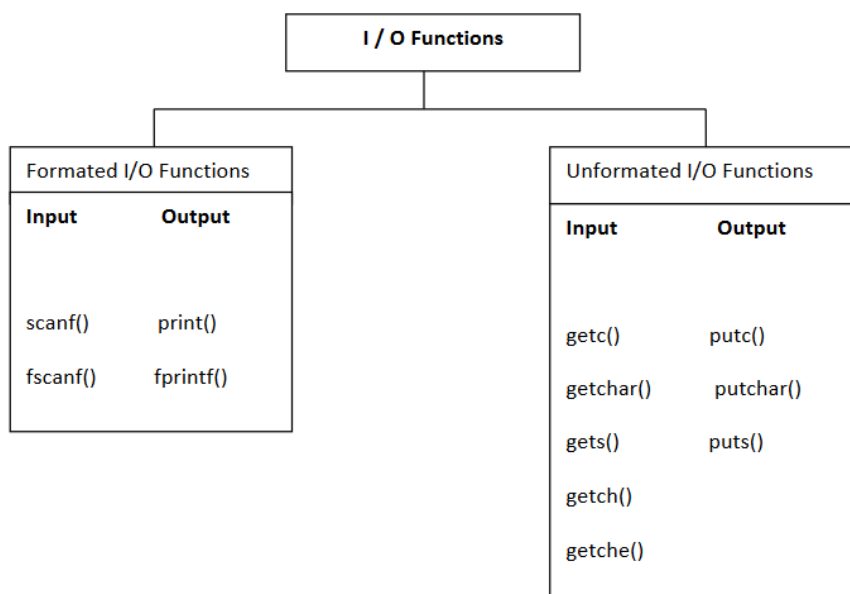
The function **putchar** prints a character each time it is called:

```
putchar(c)
```

prints the contents of the integer variable **c** as a character, usually on the screen. Calls to **putchar** and **printf** may be interleaved; the output will appear in the order in which the calls are made.

The conversion specification **%d** tells **printf** that the corresponding argument is a long integer. **printf** uses **%f** for both **float** and **double**; **%.Of** suppresses printing of the decimal point and the fraction part, which is zero.

Input / Output functions are classified into two types



Statements

A statement causes the computer to carry out some definite action. There are three different classes of statements in C:

Expression statements, Compound statements and Control statements.

Null statement

A null statement consisting of only a semicolon and performs no operations. It can appear wherever a statement is expected. Nothing happens when a null statement is executed.

Expression

Most of the statements in a C program are expression statements. An expression statement is simply an expression followed by a semicolon.

Compound statements

A compound statement (also called a "block") typically appears as the body of another statement, such as the if statement, for statement, while statement, etc.

A Compound statement consists of several individual statements enclosed within a pair of braces { }. The individual statements may themselves be expression statements, compound statements or control statements. Unlike expression statements, a compound statement does not end with a semicolon.

Control Statements

Selection Statement/Conditional Statements/Decision Making Statements

A selection statement selects among a set of statements depending on the value of a controlling expression. Or

Moving execution control from one place/line to another line based on condition. Or

Conditional statements control the sequence of statement execution, depending on the value of a integer expression.

C language supports two conditional statements.

If and switch

if Statement

The if Statement may be implemented in different forms.

- 1) simple if statement. 2) if –else statement. 3) nested if-else statement. and 4) else if ladder.

Simple if statement

The if statement controls conditional branching. The body of an if statement is executed if the value of the expression is nonzero. Or if statement is used to execute the code if condition is true. If the expression/condition is evaluated to false (0), statements inside the body of if is skipped from execution.

if the condition/expression is true, then the true statement will be executed otherwise the true statement block will be skipped and the execution will jump to the statement-x. The 'true statement' may be a single statement or group of statement.

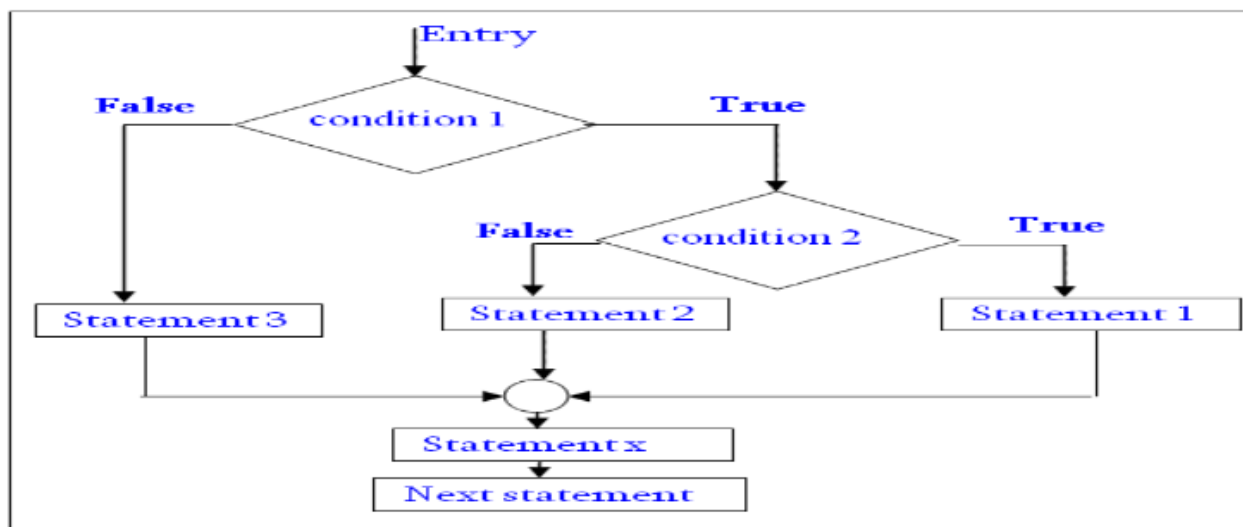
If-else statement:

The if-else statement is an extension of the simple if statement. The general form is. The if...else statement executes some code if the test expression is true (nonzero) and some other code if the test expression is false (0).

If test expression is true, codes inside the body of if statement is executed and, codes inside the body of else statement is skipped. If test expression is false, codes inside the body of else statement is executed and, codes inside the body of if statement is skipped.

Nested if-else statement

When a series of decisions are involved, we may have to use more than one if-else statement in nested form. If-else statements can also be nested inside another if block or else block or both.



If the condition-1 is false, the statement-3 and statement-x will be executed. Otherwise, it continues to perform the second test. If the condition-2 is true, the true statement-1 will be executed otherwise the statement-2 will be executed and then the control is transferred to the statement-x

Switch statement:

when there are several options and we have to choose only one option from the available ones, we can use switch statement. Depending on the selected option, a particular task can be performed. A task represents one or more statements.

Syntax:

```
switch(expression)
{
case value-1:
statement/block-1;
break;
case value-2:
statement/block-2;
break;
case value-3:
statement/block-3;
break;
case value-4:
statement/block -4;
break;
default:
default-statement/block t;
break
}
```

The expression following the keyword switch in any “C” expression that must yield an integer value. It must be an integer constant like 1,2,3...

The keyword case is followed by an integer or a character constant, each constant in each must be different from all the other.

First the integer expression following the keyword switch is evaluated. The value it gives is searched against the constant values that follow the case statements. When a match is found, the program executes the statements following the case. If no match is found with any of the case statements, then the statements following the default are executed.

Rules for writing switch () statement.

1. The expression in switch statement must be an integer value or a character constant.
2. No real numbers are used in an expression.
3. The default is optional and can be placed anywhere, but usually placed at end.
4. The case keyword must terminate with colon (:).
5. No two case constants are identical.
6. The case labels must be constants.

Iteration Statements/Loop Control Statements

A sequence of statements are executed until a specified condition is true. This sequence of statements to be executed is kept inside the curly braces { } known as the Loop body. After every

execution of loop body, condition is verified, and if it is found to be true the loop body is executed again. When the condition check returns false, the loop body is not executed.

The loops in C language are used to execute a block of code or a part of the program several times. In other words, it iterates/repeat a code or group of code many times.

Or Looping means a group of statements are executed repeatedly, until some logical condition is satisfied.

A looping process would include the following four steps.

1. Initialization of a condition variable.
2. Test the condition.
3. Executing the body of the loop depending on the condition.
4. Updating the condition variable.

C language provides three iterative/repetitive loops.

1. while loop
2. do-while loop
3. for loop

While Loop

Syntax:

variable initialization;

while (condition)

{

 statements;

 variable increment or decrement;

}

While loop can be addressed as an entry control loop. It is completed in 3 steps.

- Variable initialization. (e.g int x=0;)
- Condition (e.g while(x<=10))
- Variable increment or decrement (x++ or x--or x=x+2)

The while loop is an entry-controlled loop statement, i.e means the condition is evaluated first and it is true, then the body of the loop is executed. After executing the body of the loop, the condition is once again evaluated and if it is true, the body is executed once again, the process of repeated execution of the loop continues until the condition finally becomes false and the control is transferred out of the loop

do-while loop

Syntax:

variable initialization;

do{

 statements;

 variable increment or decrement;

}while (condition);

The do-while loop is an exit-controlled loop statement. The body of the loop is executed first and then the condition is evaluated. If it is true, then the body of the loop is executed once again. The process of execution of body of the loop is continued until the condition finally becomes false and the control is transferred to the statement immediately after the loop. The statements are always executed at least once.

For Loop

- This is an entry-controlled looping statement.
- In this loop structure, more than one variable can be initialized.
- One of the most important features of this loop is that the three actions can be taken at a time like variable initialization, condition checking and increment/decrement.
- The for loop can be more concise and flexible than that of while and do-while loops.

Syntax:

```
for(initialization; condition; increment/decrement)
{
    Statements;
}
```

Functions:

A function is a block of code/group of statements/self-contained block of statements/basic building blocks in a program that performs a particular task. It is also known as procedure or subroutine or module, in other programming languages.

To perform any task, we can create function. A function can be called many times. It provides modularity and code reusability.

Advantage of functions

1. **Code Reusability:** By creating functions in C, you can call it many times. So, we don't need to write the same code again and again.
2. **Code optimization:** It makes the code optimized we don't need to write much code.
3. **Easily to debug the program**

Types of Functions

There are two types of functions in C programming:

1. **Library Functions:** are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc. You just need to include appropriate header files to use these functions.
2. **User-defined functions:** are the functions which are created by the C programmer, so that he/she can use it many times. It reduces complexity of a big program and optimizes the code. Depending upon the complexity and requirement of the program, you can create as many user-defined functions as you want.

Function Declaration (Function Prototype).

A function declaration is the process of telling the compiler about a function name.

Syntax

```
return type function_name(parameter/argument);  
return type function-name();
```

Note: At the time of function declaration function must be terminated with;

Calling a function/function call

When we call any function control goes to function body and execute entire code.

Syntax :

```
function-name();  
function-name(parameter/argument);  
return value/ variable= function-name(parameter/argument);
```

Defining a function.

Defining of function is nothing but give body of function that means write logic inside function body.

Syntax

```
return_type function-name (parameter list) // function header.  
{  
    declaration of variables;  
    body of function; // Function  
    body return statement; (expression or value)//optional  
}
```

Function Parameters:

parameters provide the data communication between the calling function and called function. They are two types of parameters.

- i. Actual parameters.
 - ii. Formal parameters.
1. Actual Parameters: These are the parameters transferred from the calling function (main program) to the called function (function).
 2. Formal Parameters: These are the parameters transferred into the calling function (main program) from the called function(function).
- The parameters specified in calling function are said to be Actual Parameters.
 - The parameters declared in called function are said to be Formal Parameters.
 - The value of actual parameters is always copied into formal parameters.

Array

Array is a collection or group of elements (data). All the elements of array are homogeneous(similar). It has contiguous memory location.

Or An array is a data structured that can store a fixed size sequential collection of elements of same data type.

Advantage of C Array

- 1) Code Optimization: Less code to the access the data.
- 2) Easy to traverse data: By using the for loop, we can retrieve the elements of an array easily.
- 3) Easy to sort data: To sort the elements of array, we need a few lines of code only.
- 4) Random Access: We can access any element randomly using the array.

Disadvantage of Array

Fixed Size: Whatever size, we define at the time of declaration of array, we can't exceed the limit. So, it doesn't grow the size dynamically.

Where arrays are used

- to store list of Employee or Student names,
- to store marks of a students,
- or to store list of numbers or characters etc.

Declaration of an Array

data-type variable-name[size/length of array];

For example:

```
int arr[10];
```

Here int is the data type, arr is the name of the array and 10 is the size of array. It means array arr can only contain 10 elements of int type. Index of an array starts from 0 to size-1 i.e first element of arr array will be stored at arr[0] address and last element will occupy arr[9].

Initialization of an Array

After an array is declared it must be initialized. Otherwise, it will contain garbage value(any random value). An array can be initialized at either compile time or at runtime.

Compile time Array initialization

Compile time initialization of array elements is same as ordinary variable initialization.

Syntax:

data_type array_name[size]={v1,v2,...vn/list of values

eg. int age[5]={22,25,30,32,35};

Runtime Array initialization

An array can also be initialized at runtime using scanf() function. This approach is usually used for initializing large array, or to initialize array with user specified values.

Two-Dimensional Arrays

The two-dimensional array in C language is represented in the form of rows and columns, also known as matrix. It is also known as array of arrays or list of arrays. The two dimensional, three dimensional or other dimensional arrays are also known as multidimensional arrays.

Declaration of two-dimensional Array

```
data_type array_name[size1][size2];
```

Example

```
Int twodimen[2][3];
```

Pointers

Pointer is a variable that stores/hold address of another variable of same data type. it is also known as locator or indicator that points to an address of a value. A pointer is a derived data type in C

Benefit of using pointers

- Pointers are more efficient in handling Array and Structure.
- Pointer allows references to function and thereby helps in passing of function as arguments to other function.
- It reduces length and the program execution time.
- It allows C to support dynamic memory management.

Declaration of Pointer

```
data_type* pointer_variable_name;
```

```
int* p;
```

Note: void type pointer works with all data types, but isn't used often.

Initialization of Pointer variable

Pointer Initialization is the process of assigning address of a variable to pointer variable. Pointer variable contains address of variable of same data type.

Key points to remember about pointers in c:

- Normal variable stores the value whereas pointer variable stores the address of the variable.
- The content of the C pointer always be a whole number i.e. address.
- Always C pointer is initialized to null, i.e. `int *p = null`.
- The value of null pointer is 0.
- & symbol is used to get the address of the variable.
- * Symbol is used to get the value of the variable that the pointer is pointing to.
- If a pointer in C is assigned to NULL, it means it is pointing to nothing.
- Two pointers can be subtracted to know how many elements are available between these two pointers.
- But, Pointer addition, multiplication, division are not allowed.
- The size of any pointer is 2 bytes (for 16-bit compiler).

Structure

Structure is a user defined data type which hold or store heterogeneous/different types data item or element in a single variable. It is a Combination of primitive and derived data type.

A structure is a collection of one or more data items of different data types, grouped together under a single name.

Variables inside the structure are called members of structure.

Each element of a structure is called a member.

struct keyword is used to define/create a structure. struct define a new data type which is a collection of different type of data.

Syntax

```
struct structure_name /tag name
{
data_type member1;
data_type member2;
.
.
data_type member n;
};
```

Note: Don't forget the semicolon};in the ending line.

Example

```
struct employee
{
int id;
char name[50];
float salary;
};
```

Here, struct is the keyword, employee is the tag name of structure; id, name and salary are the members or fields of the structure.

Declaring structure variable

We can declare variable for the structure, so that we can access the member of structure easily.

There are two ways to declare structure variable:

- 1.By struct keyword within main() function/ Declaring Structure variables separately.
- 2.By declaring variable at the time of defining structure/ Declaring Structure Variables with Structure definition.

Accessing Structures/ Accessing members of structure

There are two ways to access structure members:

- 1) By. (member or dot operator)
- 2) By -> (structure pointer operator)

When the variable is normal type then go for struct to member operator.

When the variable is pointer type then go for pointer to member operator.